Vulnerability Information System Final Report

Authors: Chak Yui Wan (Student ID: A1870877)
Yicheng Li (Student ID: A1917736)

Supervisor: Hussain Ahmad

Date: October 26, 2025

Report submitted for
COMP SCI 3021 – Industry Project in Information Technology

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Contents

1 Introduction

1.1 Background / Context

The effective management of cybersecurity risks is a critical priority for modern organisations. Central to this effort is the Common Vulnerabilities and Exposures (CVE) system, which provides a standardised identifier for publicly known security vulnerabilities. Datasets such as the National Vulnerability Database (NVD) aggregate CVE entries with supplementary information, including severity scores (CVSS), affected software, and mitigation links. Timely access to and comprehension of this data are essential for IT professionals and security teams to conduct risk assessments and prioritise remediation efforts. However, the official presentation of this data is typically dense, highly technical, and optimised for in-depth analysis rather than rapid assessment by a broader range of stakeholders.

1.2 Motivation

This project was motivated by the need for a more accessible and intuitive interface for querying and visualising CVE information. For business clients or IT professionals who are not full-time security analysts, parsing the extensive technical data on standard databases can be time-consuming and cognitively demanding. A significant gap exists for a streamlined tool that can rapidly retrieve, parse, and present the most critical CVE details—such as severity, impact, and affected systems—in a clear, concise, and visually engaging format. This project addresses the demand for a "quick look" solution, enabling users to perform rapid initial assessments without being overwhelmed by technical minutiae.

1.3 Proposed Solution

To address these challenges, the proposed solution is a full-stack web application built on a three-tier architecture. The presentation layer (front-end) is a static web interface developed with HTML, CSS, and client-side JavaScript, designed to present complex vulnerability data in an intuitive, newspaper-style layout. The application layer (backend) is a custom API developed using Node.js and Express.js. This API serves two primary functions: (1) It queries a local MySQL database to retrieve structured CVE data based on user searches; and (2) It provides an AI-augmented chat service by securely interfacing with the Anthropic Claude 3.5 Sonnet large language model (LLM), passing the relevant CVE data as context for analysis. The data layer consists of a local MySQL database containing a curated $CVE_2024 table, which stores the pre-processed vulnerability information for rapid retrieval.$

1.4 Key Contributions

The primary contributions of this project are:

- Designed and implemented a custom Node.js backend API and a relational MySQL database schema to efficiently store, query, and retrieve specific CVE data.
- Developed a highly visual and intuitive front-end interface that transforms complex CVE data into accessible visualisations, including interactive timelines, CVSS score gauges, and risk metric charts.
- Integrated a generative AI-powered chat assistant (Anthropic Claude 3.5 Sonnet) that leverages Retrieval-Augmented Generation (RAG) to provide users with contextual, natural-language explanations and analysis of the selected vulnerability.

2 Methodology / Approach

The project was executed following a structured, five-phase agile methodology, moving from initial concept to a deployment-ready application. This approach ensured that client requirements were accurately translated into functional software through iterative stages of design, development, and validation.

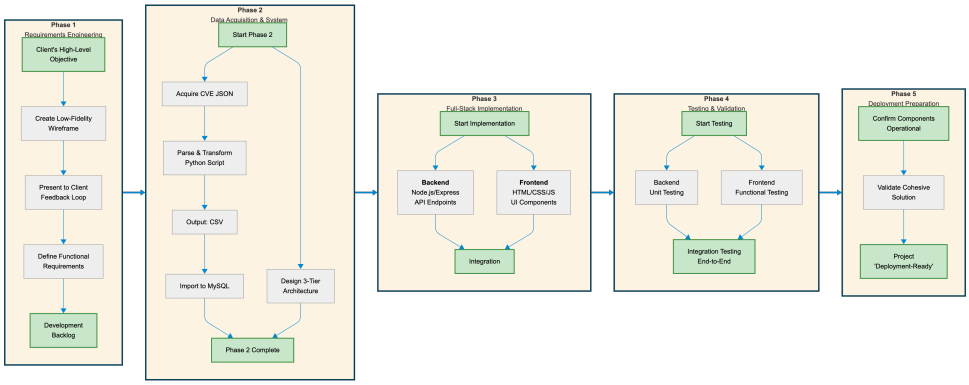

Figure 1: A flowchart summarizing the project methodology.

2.1 Requirements Engineering and Prototyping

The initial phase involved translating the client's high-level objective—a streamlined CVE search website—into a tangible concept. A low-fidelity, hand-drawn wireframe was created to establish a shared visual understanding of the proposed layout and user journey. This prototype was presented to the client, facilitating a feedback loop that resulted in a concrete set of functional requirements. These requirements were formally defined and categorized into five key visualisation features (e.g., 'CVE Detail Panel', 'Severity Highlight') and six core interaction functions (e.g., 'Complete Search Functionality', 'Search History Cache'), which subsequently served as the development backlog.

2.2 Data Acquisition and System Design

This phase focused on establishing the data persistence layer and the overall system architecture. The complete 2024 CVE dataset was first acquired in its raw JSON format from the official cve.org repository. Due to the complexity of the nested JSON structure, a custom Python script was developed to parse the data, extract the essential fields (e.g., ID, description, scores, vendors), and transform it into a flattened CSV format. This structured CSV file was then imported into a local MySQL database, which defined the schema for the `CVE_2024` table. Concurrently, the 3-tier application architecture was designed, separating the front-end (browser), the backend API (Node.js), and the data layer (MySQL).

2.3 Full-Stack Implementation

With the architecture and data schema in place, implementation was divided between the two team members to enable parallel development:

- **Backend Development (Ian):** This workstream focused on building the Node.js/Express server. It involved creating the database connection, writing the SQL queries, and defining the API endpoints (e.g., `/api/cve/:cveId` and `/api/cve-chat`) required to serve data to the client application.
- **Frontend Development (Yicheng):** This workstream focused on translating the prototype into a functional user interface. This involved writing the HTML structure, styling with CSS, and developing the client-side JavaScript (`Enquiry-Search.js`) to handle user input, make asynchronous API calls, and dynamically render the response data into the defined visual components.

2.4 Testing and Validation

A multi-stage testing strategy was employed to ensure application correctness and stability.

- **Backend Unit Testing:** The API endpoints were tested directly by accessing their local URLs (e.g., `http://localhost:3000/api/cve/1001`) to verify correct JSON responses, proper error handling for invalid inputs, and successful database connectivity.
- **Frontend Functional Testing:** The user interface was manually tested to confirm that all specified interaction requirements (e.g., 'Enter Key to Search', 'Loading Spinner') functioned as expected.
- **Integration Testing:** End-to-end tests were performed to ensure the front-end JavaScript correctly fetched data from the backend API and that the data was accurately displayed in the visual components.

2.5 Deployment Preparation

In the final phase, the application was prepared for demonstration. While not deployed to a public cloud-hosted service, the project was confirmed to be fully 'deployment-ready'. All components (front-end, backend API, and database) were fully configured and operational within a local development environment, validating that the entire stack functions as a cohesive solution.

3 Experimental Evaluation

This section details the testing environment used to validate the application and presents the final results, analysing how the implemented solution successfully meets the project's objectives.

3.1 Experimental Setup

The application was tested in a comprehensive, multi-platform local environment to ensure robustness and cross-browser compatibility. The testing setup was configured as follows:

- **Operating Systems:** macOS 26 Tahoe and Windows 11.
- **Web Browsers:** Google Chrome (`v128`), Mozilla Firefox (`v129`), Microsoft Edge (`v128`), and Apple Safari (`v18`).
- **Backend Environment:** Node.js `v24.2.0`.
- **Database Server:** MySQL (`v8.0`).

This setup allowed for the validation of the front-end rendering, API connectivity, and backend logic across all major user platforms.

3.2 Results and Analysis

The project successfully translated the initial low-fidelity concept into a fully functional and visually rich web application.

3.2.1 Analysis of Core Visualisation (Prototype vs. Final)

Figure **??** shows the initial, hand-drawn prototype that served as the foundational blueprint for the user interface. It established the core layout, including a main search, a timeline, a CVSS score gauge, and various metric panels. Figure **??** presents the final implemented user interface, demonstrating a high-fidelity realisation of the initial concept. The final design effectively translates the prototype's wireframes into polished components:

- **CVSS Score Analysis:** The simple gauge from the prototype was developed into a dynamic, colour-coded SVG gauge (displaying 7.2 in the example) that immediately communicates the 'HIGH PRIORITY' status.
- **Timeline Information:** The conceptual monthly timeline was implemented as a clean, twelve-month visualisation, clearly marking the 'Published' (Jan 2024) and 'Last Modified' (Nov 2024) dates.
- **Risk Metrics:** The 'Attack Profile' and 'Risk Metrics' tables were implemented as distinct components with clear labels (e.g., Exploitability, Impact) and horizontal bar charts, providing a quick-look assessment of risk factors.
- **Comprehensive Threat Analysis:** The conceptual radar chart was successfully implemented, providing a multi-dimensional view of the threat by plotting metrics for CVSS, Exploitability, Impact, and Trending.

This direct comparison validates that the core visualisation requirements were met, transforming raw data into an accessible, newspaper-style report.

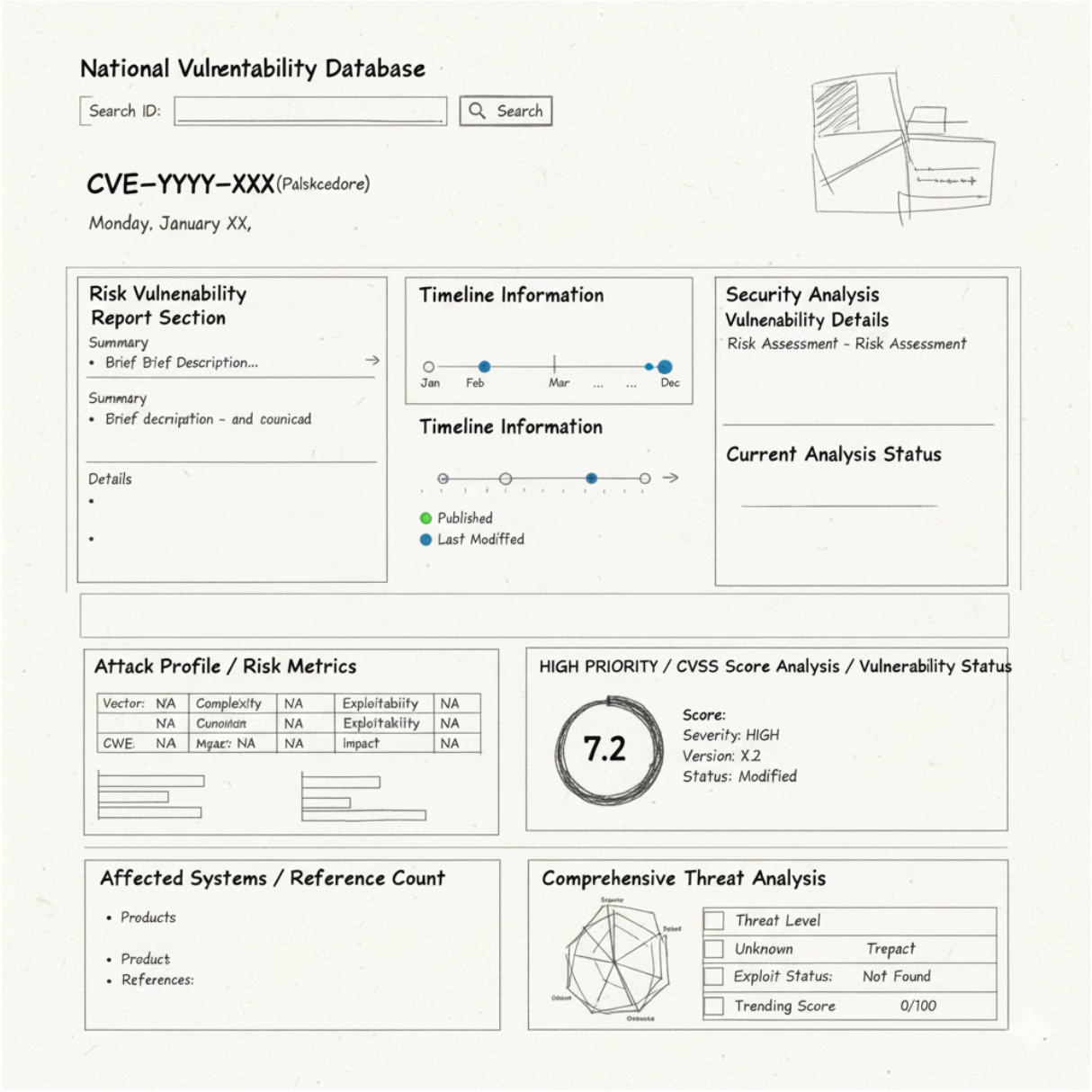

Figure 2: Initial Low-Fidelity Prototype

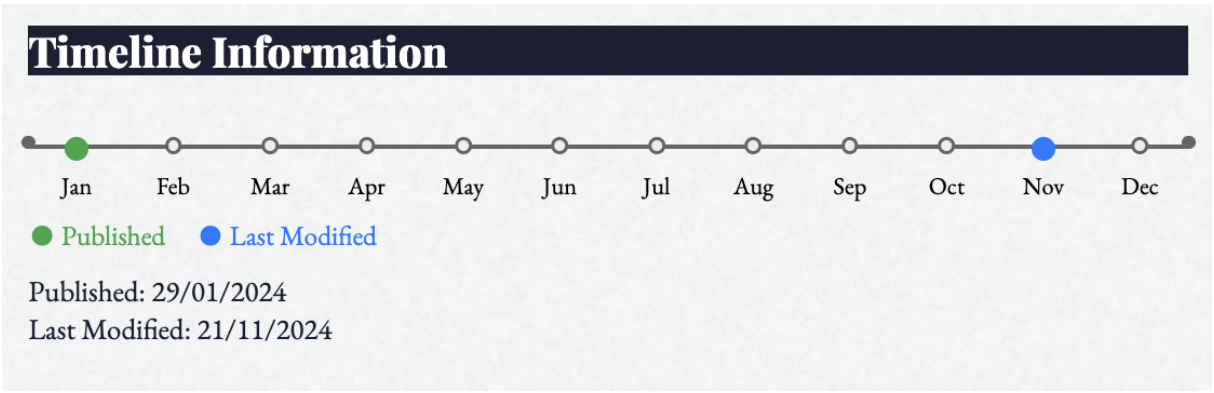

Figure 3: Detailed Implementation of the 12-Month Vulnerability Timeline

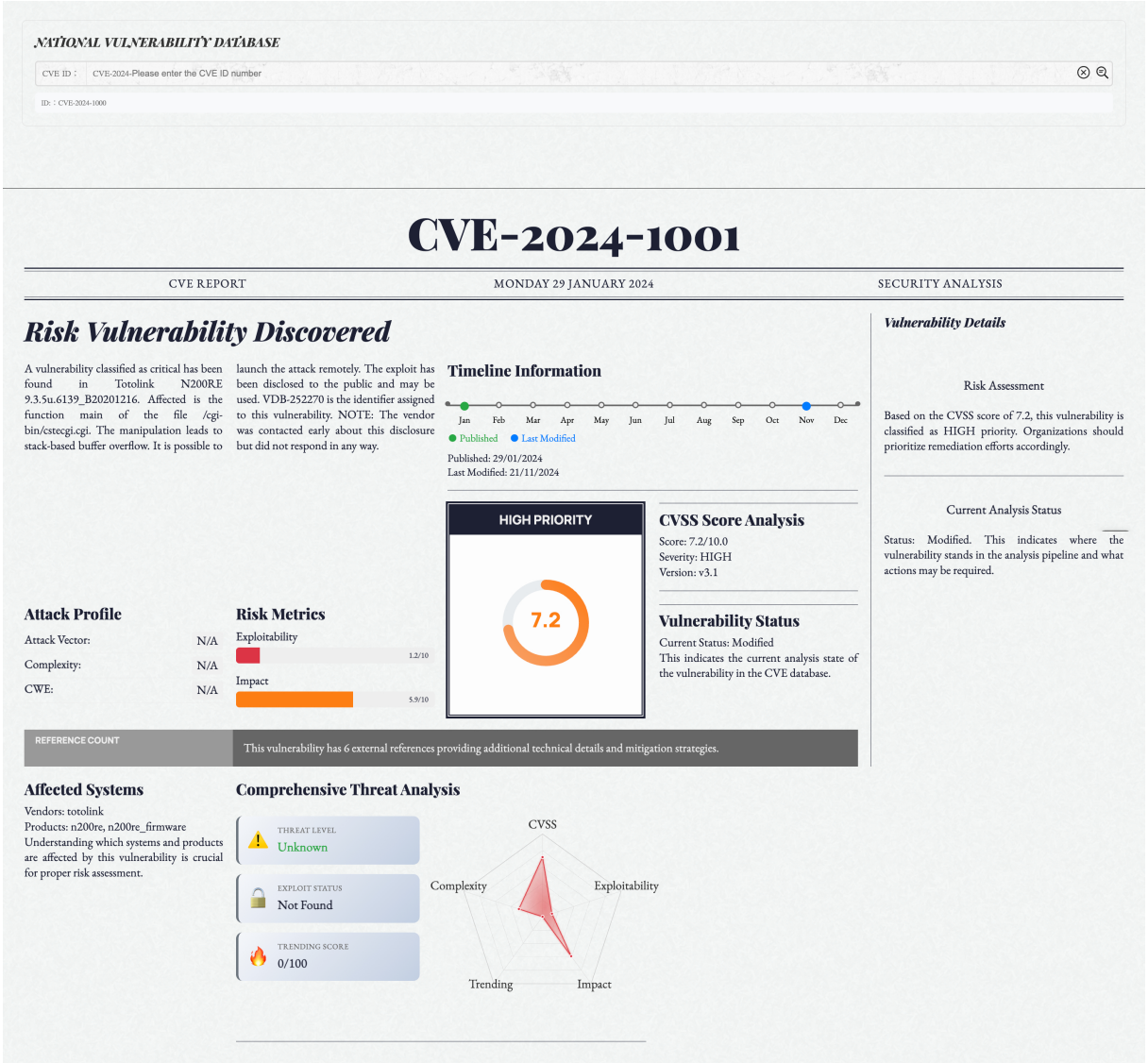

Figure 4: Final Implemented Results Interface

3.2.2 Analysis of Advanced Interaction (AI Assistant)

Beyond the static display of data, the project incorporated an advanced interaction feature: a generative AI assistant. As shown in Figure **??**, the 'CVE AI Assistant' is context-aware, loading with information specific to the CVE being viewed (e.g., `CVE-2024-1001`). It prompts the user with relevant questions, such as "How severe is this vulnerability?" and "What systems are affected?". This feature provides significant added value, allowing users to move beyond simple data consumption and engage in a natural-language dialogue to analyse the vulnerability's implications.

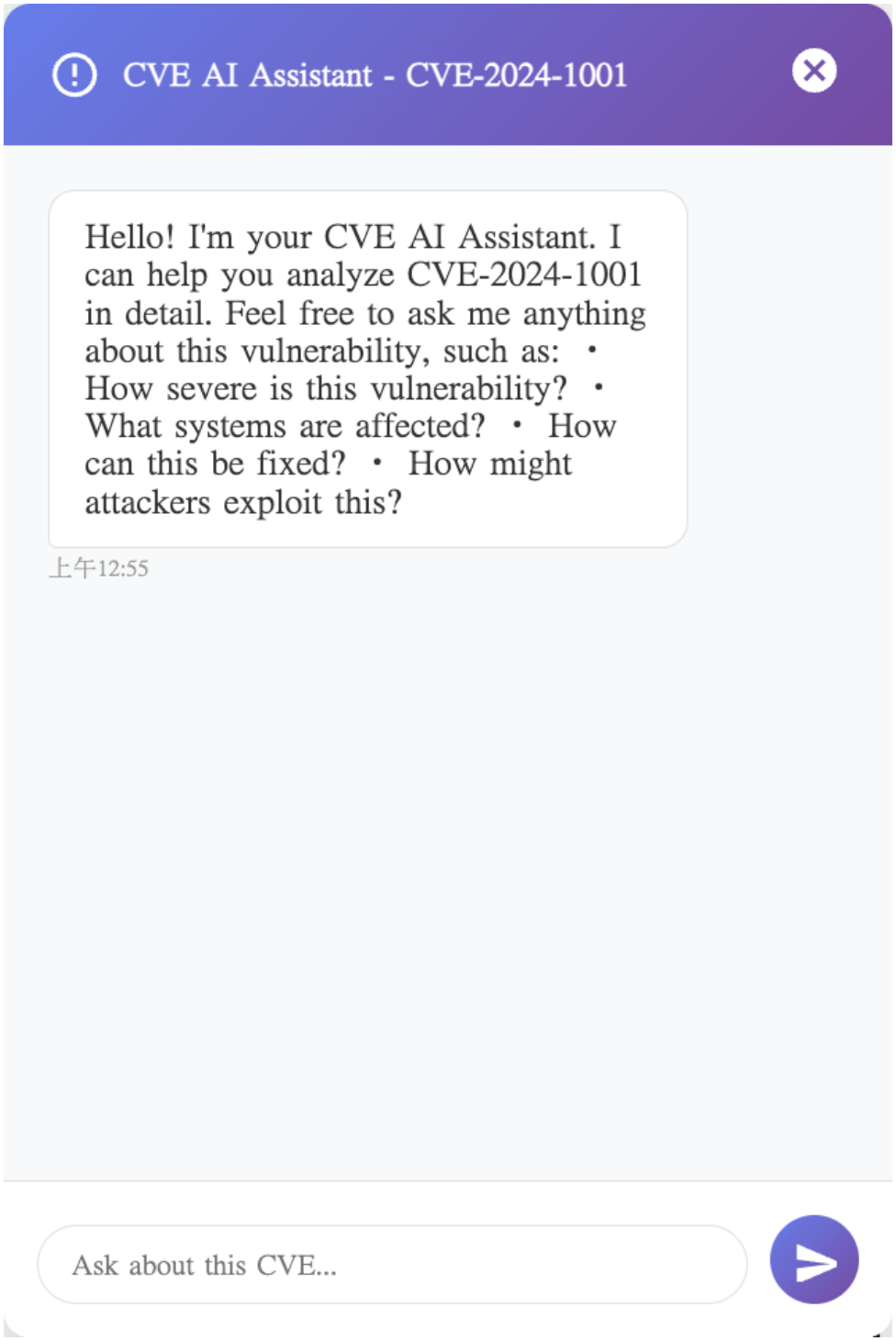

Figure 5: Context-Aware AI Chat Assistant

4 Discussion

This section reflects on the project's outcomes, key challenges faced during development, and the practical implications of the solution.

4.1 Project Outcomes and Successes

The project was highly successful in achieving its primary goal: transforming dense, technical CVE data into an accessible, user-friendly format. The most effective aspect of the project was the final UI/UX design. By simulating a "newspaper style" layout, the application provides a familiar and intuitive reading experience. This design choice directly addresses the client's "quick look" requirement, as users can rapidly scan headings, gauges, and charts to assess a vulnerability's severity, similar to how one would read a newspaper. The integration of multiple visualisation types—such as the 12-month timeline, CVSS gauge, and risk radar chart—proved to be an effective strategy for converting abstract text values into easily understandable information.

4.2 Challenges Encountered

Several challenges emerged during the development process, primarily related to data processing and design decisions:

- **Data Ambiguity and Processing:** The raw CVE data, acquired as a bulk JSON download, was unexpectedly complex. A significant challenge was parsing this data, as a single CVE entry often contained multiple data objects representing different CVSS versions (v2, v3, v4). Deciding which version to prioritise and developing the Python script to effectively parse and flatten this nested, and at times redundant, data into a clean database schema was a major hurdle. Ultimately, CVSS v3 was selected as the standard for display.
- **Visualisation Design:** A core development challenge was determining the most effective method to visualise abstract data. The team was committed to a highly visual approach, but this required careful consideration of which chart types (e.g., bar, radar, gauge) best represented specific metrics. For example, designing the timeline as a literal 12-month calendar view was a deliberate choice to enhance clarity, but it required more design and implementation effort than a simple text-based date list.

4.3 Practical Implications

The practical utility of this project is clear. It serves as an effective "first-response" tool for IT professionals, managers, and other stakeholders who need to understand a vulnerability's risk without performing a deep technical dive. By providing a consolidated, high-level dashboard, it accelerates the triage and risk assessment process. Furthermore, the integration of the Claude-powered AI assistant demonstrates a viable path toward Retrieval-Augmented Generation (RAG) in security tools, allowing even non-experts to ask complex questions and receive immediate, context-aware analysis.

5 Limitations

- **Technical Constraint:** (e.g., We were limited by...) Aliquam lectus. Vivamus leo. Quisque ornare tellus ullamcorper nulla. Mauris porttitor pharetra tortor. Sed fringilla justo sed mauris. Mauris tellus. Sed non leo. Nullam elementum, magna in cursus sodales, augue est scelerisque sapien, venenatis congue nulla arcu et pede. Ut suscipit enim vel sapien. Donec congue. Maecenas urna mi, suscipit in, placerat ut, vestibulum ut, massa. Fusce ultrices nulla et nisl.

- **Time Limitation:** (e.g., Due to the project timeline, we could not...) Etiam ac leo a risus tristique nonummy. Donec dignissim tincidunt nulla. Vestibulum rhoncus molestie odio. Sed lobortis, justo et pretium lobortis, mauris turpis condimentum augue, nec ultricies nibh arcu pretium enim. Nunc purus neque, placerat id, imperdiet sed, pellentesque nec, nisl. Vestibulum imperdiet neque non sem accumsan laoreet. In hac habitasse platea dictumst. Etiam condimentum facilisis libero. Suspendisse in elit quis nisl aliquam dapibus. Pellentesque auctor sapien. Sed egestas sapien nec lectus. Pellentesque vel dui vel neque bibendum viverra. Aliquam porttitor nisl nec pede. Proin mattis libero vel turpis. Donec rutrum mauris et libero. Proin euismod porta felis. Nam lobortis, metus quis elementum commodo, nunc lectus elementum mauris, eget vulputate ligula tellus eu neque. Vivamus eu dolor.

- **Scope Limitation:** (e.g., The project scope did not include...) Nulla in ipsum. Praesent eros nulla, congue vitae, euismod ut, commodo a, wisi. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Aenean nonummy magna non leo. Sed felis erat, ullamcorper in, dictum non, ultricies ut, lectus. Proin vel arcu a odio lobortis euismod. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Proin ut est. Aliquam odio. Pellentesque massa turpis, cursus eu, euismod nec, tempor congue, nulla. Duis viverra gravida mauris. Cras tincidunt. Curabitur eros ligula, varius ut, pulvinar in, cursus faucibus, augue.

6 Conclusion

example_website Nulla mattis luctus nulla. Duis commodo velit at leo. Aliquam vulputate magna et leo. Nam vestibulum ullamcorper leo. Vestibulum condimentum rutrum mauris. Donec id mauris. Morbi molestie justo et pede. Vivamus eget turpis sed nisl cursus tempor. Curabitur mollis sapien condimentum nunc. In wisi nisl, malesuada at, dignissim sit amet, lobortis in, odio. Aenean consequat arcu a ante. Pellentesque porta elit sit amet orci. Etiam at turpis nec elit ultricies imperdiet. Nulla facilisi. In hac habitasse platea dictumst. Suspendisse viverra aliquam risus. Nullam pede justo, molestie nonummy, scelerisque eu, facilisis vel, arcu.

Curabitur tellus magna, porttitor a, commodo a, commodo in, tortor. Donec interdum. Praesent scelerisque. Maecenas posuere sodales odio. Vivamus metus lacus, varius quis, imperdiet quis, rhoncus a, turpis. Etiam ligula arcu, elementum a, venenatis quis, sollicitudin sed, metus. Donec nunc pede, tincidunt in, venenatis vitae, faucibus vel, nibh. Pellentesque wisi. Nullam malesuada. Morbi ut tellus ut pede tincidunt porta. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Etiam congue neque id dolor.

7 Replication Package

The complete source code, documentation, and setup instructions for this project are available at the following publicly accessible GitHub repository: https://github.com/ianian-c-y/COMP-SCI-3021---Industry-Project-in-Information-Technology The repository includes a `README.md` file with detailed instructions for setting up the environment and running the system.